

Abuse Frame Based Security Analysis and Validation of a Replicated Note Storage System

[Group 17 – Abuse Frames–Based Security Requirements Analysis](#)

IDMANSOUR Salma
GARTANI Othmane
CRISTEA Ana
ILIESCU Miruna-Elena

Table of Contents

1. Executive Summary and Project Context.....	3
2. Base Problem Frame Analysis.....	3
2.1 Principal Domains.....	3
2.2 Functional Requirements.....	3
2.3 Correctness Argument.....	4
3. Security Requirements Analysis Using Abuse Frames.....	4
4. Identified Abuse Frames and Anti-Requirements.....	4
4.1 AR-1: Unauthorized Note Disclosure (Interception).....	4
4.2 AR-2: Unauthorized Note Modification (Modification).....	5
4.3 AR-3: Lock Starvation Attack (Denial of Access).....	5
4.4 AR-4: Implementation and Validation of AR-4 (Cross-Origin Token Theft).....	6
4.5 AR-5: Replication Desynchronization (Modification / Denial of Access).....	7
5. Security Machine Specification and Countermeasures.....	7
5.1 Authentication and Authorization.....	7
5.2 Locking Mechanism.....	8
5.3 Cross-Origin Security.....	8
5.4 Replication Integrity.....	8
5.5 Security Test Results.....	8
6. Robustness Testing and Abuse Frame Arguments.....	10
7. Implementation Overview.....	10
8. Conclusion and Security Assessment.....	10

1. Executive Summary and Project Context

This project concerns the design and implementation of a secure software platform for storing and accessing personal textual notes. The system supports multiple users, ensures strict isolation between users' data, and enables controlled sharing of notes in read-only or read/write modes. To improve resilience and availability, the system relies on two servers that replicate note storage.

Our group implemented a file-based storage system exposed through a RESTful API, accessed by a web-based front-end hosted on a different domain. This architectural choice introduces additional security concerns, particularly related to cross-origin access, authentication, and data leakage. The system therefore integrates security mechanisms aimed at preserving confidentiality, integrity, and availability (CIA) of notes, even in the presence of malicious attackers.

As required, this report applies the Abuse Frames approach (Lin et al., 2003) to systematically analyse security threats, identify vulnerabilities, and justify the implemented countermeasures.

2. Base Problem Frame Analysis

Before analysing security threats, the functional scope of the system is bounded using a Base Problem Frame, following Jackson's Problem Frames method.

2.1 Principal Domains

- Machine (M): The *RESTful Note Server*, responsible for handling HTTP requests, enforcing access control, managing locks, and interacting with storage.
- Lexical Domain (X): The *File Storage*, which contains textual note files and associated metadata (ownership, sharing permissions, locks).
- Biddable Domain (B): The *User*, who issues requests to create, read, edit, and share notes. User behaviour is not predictable and cannot be controlled by the machine.
- Causal Domains (C): Supporting services such as authentication modules and replication mechanisms, whose behaviour follows defined causal rules.

2.2 Functional Requirements

The main functional requirements (Req) are:

- Users can create, read, update, and delete their own notes.
- Users cannot access notes belonging to other users.
- Notes can be shared with other users in read-only or read/write mode.
- Write access is allowed only when a note is explicitly locked by a single user.
- Notes remain accessible despite single-server failures.

2.3 Correctness Argument

The base correctness argument is expressed as:

$$S, W \rightarrow R$$

Where:

- S is the specification of the RESTful server,
- W is the bounded problem world (users, storage, network),
- R is the set of functional requirements.

If the REST server correctly processes authenticated requests and the file system reliably stores textual data, then users observe their notes correctly displayed and updated according to their actions.

3. Security Requirements Analysis Using Abuse Frames

Following Lin et al. (2003), security threats are analysed by introducing anti-requirements (AR)—the goals of malicious attackers—and modelling them using Abuse Frames. Each abuse frame is composed with the base problem frame to identify vulnerabilities.

Security threats are classified according to the three standard abuse frame classes:

- Interception (Confidentiality)
- Modification (Integrity)
- Denial of Access (Availability)

4. Identified Abuse Frames and Anti-Requirements

4.1 AR-1: Unauthorized Note Disclosure (Interception)

Anti-Requirement (AR-1):

An attacker obtains the textual content of a private note without authorization.

[Abuse Frame Description:](#)

- Asset (AS): Note content stored in the file system.
- Malicious Machine (M/M): A crafted REST client or intercepting proxy.

- Attacker (AT): External or internal malicious user.

Vulnerability Conditions:

- *uAPI*: REST endpoints do not strictly verify note ownership.
- *uID*: User identity information can be spoofed via request headers if not bound to authenticated JWT identity.

$$v_1 = \{uAPI, uID\}$$

4.2 AR-2: Unauthorized Note Modification (Modification)

Anti-Requirement (AR-2):

An attacker modifies the content of a note they do not own or edits a note without holding a valid lock.

Abuse Frame Description:

- Asset: Note files and symbolic note states.
- Malicious Machine: Forged PUT/POST REST requests.

Vulnerability Conditions:

- *uLock*: Write operations are allowed without an active lock.
- *uACL*: Write permissions are not correctly enforced.

$$v_2 = \{uLock, uACL\}$$

4.3 AR-3: Lock Starvation Attack (Denial of Access)

Anti-Requirement (AR-3):

An attacker prevents legitimate users from editing a note by holding a lock indefinitely.

Abuse Frame Description:

- Asset: Lock state associated with a note.
- Malicious Machine: Repeated lock acquisition without release.

Vulnerability Conditions:

- *uTTL*: Locks have no expiration time.
- *uUser*: Any authenticated user can acquire locks repeatedly.

$$v_3 = \{uTTL, uUser\}$$

4.4 AR-4: Implementation and Validation of AR-4 (Cross-Origin Token Theft)

Goal (AR-4). Prevent a malicious web front-end (different origin) or injected JavaScript from stealing authentication material and using it to access private notes. The associated vulnerability set is $v_4 = \{uJWT, uCORS\}$, where $uJWT$ corresponds to browser-accessible token storage and $uCORS$ to weak cross-origin access control.

Countermeasures implemented.

1. HttpOnly Cookie Authentication (mitigates $uJWT$).
A dedicated endpoint `POST /auth/login_cookie` was introduced. On successful login, the server sets an authentication cookie:
 - `HttpOnly` (not accessible to JavaScript),
 - `SameSite=Strict` (reduces cross-site cookie sending),
 - `Path=.`
This prevents a malicious script from reading the token through `document.cookie`.
2. Strict CORS Allowlist (mitigates $uCORS$).
The API enforces an explicit origin allowlist (e.g., `http://127.0.0.1:5173`, `http://localhost:5173`). Requests originating from non-authorized domains do not receive `Access-Control-Allow-Origin`, so browsers block malicious front-ends from reading API responses.

Automated test (pytest) added for AR-4.

A new test `test_ar4_cookie_is_httponly_and_cors_is_strict_allowlist` validates both properties:

- The response of `POST /auth/login_cookie` includes a `Set-Cookie` header containing `HttpOnly` and `SameSite=Strict`.
- A legitimate origin receives `Access-Control-Allow-Origin`, while a malicious origin (e.g., `https://evil.example`) does not.

Manual browser demonstration (optional but reproducible).

- After logging in via cookie on the legitimate front-end, `document.cookie` does not reveal

the authentication token (HttpOnly).

- A separate “evil” front-end hosted on another port/origin attempts `fetch("http://127.0.0.1:8000/notes", {credentials:"include"})`; the browser blocks it (CORS), preventing disclosure of private notes.

4.5 AR-5: Replication Desynchronization (Modification / Denial of Access)

Anti-Requirement (AR-5):

An attacker causes inconsistent or corrupted note states between replicated servers by injecting, replaying, or omitting replication events.

Abuse Frame Description:

- Asset : Integrity and consistency of the replication event log.
- Malicious Machine : A forged replication client or man-in-the-middle component.
- Attacker : External attacker or compromised server attempting to tamper with replication.

Vulnerability Conditions:

1. *uRepAuth*: Replication messages are not authenticated.
2. *uOrder*: Event ordering or completeness is not guaranteed.
 $v_5 = \{uRepAuth, uOrder\}$

5. Security Machine Specification and Countermeasures

The Security Machine represents the set of mechanisms designed to prevent abuse frame realisation.

5.1 Authentication and Authorization

- JWT-based authentication ensures reliable user identification.
- Access Control Lists (ACLs) enforce ownership and sharing permissions.

These measures invalidate vulnerabilities *uAPI* and *uACL*, blocking AR-1 and AR-2.

5.2 Locking Mechanism

- A dedicated locking module ensures exclusive write access.
- Locks are associated with a single user and note.
- Lock expiration (TTL) prevents indefinite lock holding.

These measures eliminate *uLock* and *uTTL*, blocking AR-2 and AR-3.

5.3 Cross-Origin Security

- Strict CORS policies restrict allowed origins.
- Authentication is based on Bearer JWT tokens validated server-side.
- Client-side token storage is protected through front-end controls and Content Security Policy (CSP), reducing the risk of cross-origin token theft.

These measures remove *uJWT* and *uCORS*, blocking AR-4.

5.4 Replication Integrity

- Event logs are exchanged only between authenticated servers.
- Events are ordered and validated before application.

These measures remove *uRepAuth* and *uOrder*, blocking AR-5.

5.5 Security Test Results

To validate the robustness of the proposed Security Machine against the identified Abuse Frames, two complementary security test suites were implemented.

- The first suite, `test_security_interception.py`, provides an integration-level validation of confidentiality and access-control guarantees. It exercises end-to-end REST API flows in which a malicious user attempts to intercept or infer another user's notes through unauthorized read, modification, listing, or sharing operations. These tests cover multiple Interception and Modification abuse scenarios, including direct note access by identifier, metadata disclosure through listing endpoints, identity spoofing via manipulated request headers, and misuse of sharing mechanisms. Successful execution of this suite demonstrates that the system consistently enforces user isolation and sharing policies, thereby refuting AR-1 (Unauthorized Disclosure) and AR-2 (Unauthorized Modification via Sharing Abuse).
- The second suite, `test_anti_requirements.py` is explicitly requirements-driven and directly targets the formal anti-requirements derived from the Abuse Frame analysis. Tests in this suite verify that authentication and authorization mechanisms cannot be bypassed through header spoofing when a valid JWT is present, that expired tokens are correctly

rejected, and that intercepted lock identifiers cannot be reused to perform unauthorized modifications. In addition, replication-specific attacks are exercised by validating replication-token enforcement, replay detection, and event ordering guarantees, while lock-expiration tests ensure that write locks cannot be abused to cause denial of access. Together, these tests empirically demonstrate that the Security Machine prevents the satisfaction of AR-1 (Confidentiality), AR-2 (Integrity), AR-3 (Availability through lock starvation), and AR-5 (Replication Integrity).

All security tests completed successfully. For each tested scenario, malicious requests failed to satisfy their corresponding anti-requirements, providing concrete evidence that $MS, v(W) \rightarrow AR$ holds for the identified classes of attacks.

The table below maps each targeted security test to the corresponding anti-requirement identified in the Abuse Frame analysis, illustrating how specific attacker goals are empirically refuted by the implemented Security Machine.

Test	Anti-Requirement	Threat Class
test_ar1_jwt_precedence_over_header	AR-1 (Confidentiality)	Interception (Identity Spoofing)
test_ar1_jwt_owner_ignores_spoofed_header	AR-1 (Confidentiality)	Interception (Authentication Bypass)
test_expired_jwt_is_rejected	AR-1 (Confidentiality)	Interception (Token Replay / Expiry Abuse)
test_ar2_cannot_modify_note_without_owning_lock	AR-2 (Integrity)	Modification (Lock Reuse / Tampering)
test_ar3_lock_starvation_expires_and_allows_reacquire	AR-3 (Availability)	Denial of Access (Lock Starvation)
test_replicate_events_requires_token_and_applies_with_valid_token	AR-5 (Replication Integrity)	Interception / Injection

test_ar5_replication_replay_and_ordering	AR-5 (Replication Integrity)	Replay / Ordering Abuse
--	------------------------------	-------------------------

6. Robustness Testing and Abuse Frame Arguments

For each anti-requirement AR, robustness is demonstrated by showing:

$$MS, v(W) \rightarrow AR$$

That is, even if an attacker attempts to exploit the vulnerability conditions, the Security Machine prevents satisfaction of the anti-requirement.

Examples:

- Unauthorized cross-user note access results in 404 Not Found (to prevent information leakage), while requests missing valid authentication are rejected with 401 Unauthorized.
- Write attempts without a valid lock are rejected.
- Cross-origin requests from untrusted domains are blocked.

7. Implementation Overview

- Back-end: RESTful API managing notes, locks, ACLs, and replication.
- Storage: File-based note storage with metadata.
- Replication: Event-based synchronization between two servers.
- Front-end: Web application hosted on a separate domain with strict CORS enforcement.

8. Conclusion and Security Assessment

By applying the Abuse Frames approach, security threats were analysed systematically from the attacker's perspective. The identification of explicit anti-requirements and vulnerability conditions enabled the derivation of precise and justified security countermeasures.

The final system enforces strong confidentiality, integrity, and availability guarantees within a clearly bounded problem context. Trust assumptions are limited to the underlying operating system and authenticated replication channels. Overall, the Abuse Frames method proved effective for early and structured security requirements engineering in this project.

The effectiveness of the proposed security design is further supported by automated security

tests that empirically demonstrate that identified anti-requirements cannot be satisfied under realistic attack scenarios.